

TOI 20th at SU

Report from Scientific Committee

Contest Report

PROBLEM REPORT

การเปลี่ยนแปลงแนวทางข้อสอบ

- เพิ่มเวลาการสอบเป็น 5 ชม.
 - เพื่อให้สามารถออกโจทย์ที่ต้องใช้เวลาทำได้มากขึ้น
 - ตอนนี้ใช้เวลาเท่า IOI แล้ว
 - มีข้อที่มีการให้คะแนนแบบ partial scoring
 - มีข้อที่มีการให้คะแนนแบบ formula scoring
 - ข้อ routes
 - มีการใช้ข้อสอบแบบส่งเฉพาะฟังก์ชัน
 - ปีหน้าจะสามารถออก “โจทย์แบบ interactive” ได้แล้ว
- ถ้า $S \geq T$ ผู้เข้าแข่งขันจะได้คะแนนเต็ม 100 ในชุดทดสอบนั้น
 - ถ้า $S < T$ ผู้เข้าแข่งขันจะได้คะแนนเป็น $\frac{S^2}{T^2} \times 100$

แนวทางในการทำข้อสอบ

- ถ้าคิด full solution ไม่ได้ อยากรู้ให้พยายามทำทีละ subtask

Time	Task	Status
03:12:4	<u>d2p1 shopping</u>	▶ Scored (0.0 / 100.0)
02:58:2	<u>d2p1 shopping</u>	▶ Scored (17.0 / 100.0)
02:25:1	<u>d2p1 shopping</u>	▶ Scored (8.0 / 100.0)
02:15:0	<u>d2p1 shopping</u>	▶ Scored (13.0 / 100.0)
02:13:5	<u>d2p1 shopping</u>	▶ Scored (13.0 / 100.0)
02:13:4	<u>d2p1 shopping</u>	▶ Compilation failed

Subtask 3

Subtask 2

Subtask 1

ได้คะแนนรวม 38

Solution Count

มีกรรมการที่
คนช่วยทำ

เฉลยที่ได้คะแนน
บางส่วนมีอีกอัน

Problem	Official Solution	Subtask solution	#Subtask
D1: Orbs	5	10	7
D1: Route	2	9	5 (variable)
D1: Sculpture	2	5	6 (partial)
D2: Bit String	3	15	6
D2: Tour	2	12	7
D2: Lover	1	6	4
--: Art Toy	3	7	6

โอกาสได้คะแนน
ย่อย ๆ

Orbs

- Summary

- โจทย์ใช้ Data Structure, sorting

- Basic Solution $O(L * N \lg N)$

- Simulate การทำงานของเครื่องจักร โดยตรง (ต้องหาตัวลำดับที่ a และ b ในแต่ละรอบโดยการ sort)

- Subtask 2 Solution $O(L * N)$

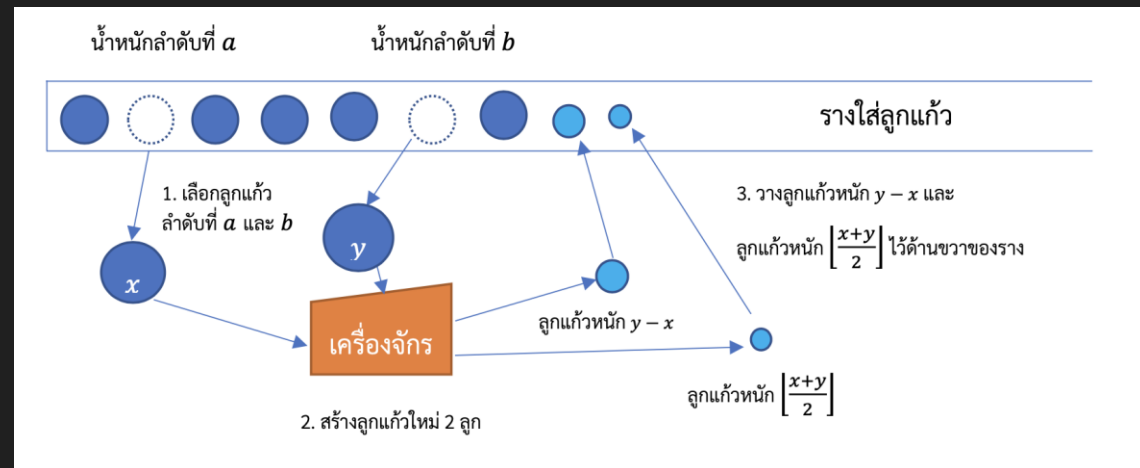
- Simulate การทำงานของเครื่องจักร โดยตรง (ต้องหาตัวลำดับที่ a และ b ในแต่ละรอบโดย $O(N)$)

- Subtask 3 Solution

- เขียน Data Structure (พวก tree ต่าง ๆ) เอง

- Subtask 4,5

- หยิบ min, max เสมอ ใ้ว่าถ้าหาทางหาได้เร็ว ๆ ว่าตัวที่ a, b อยู่ตรงไหนก็น่าจะได้



Orbs

- Official $O(L * \lg N)$

- สร้าง tree (ใช้ multiset ก็ได้) 3 ต้น
- มาเก็บ a ตัวแรก, (b-a) ตัวถัดมา และ $n - b$ ตัวสุดท้าย
- ตัวที่ a, b คือ max ของต้นแรกและต้นสอง หาเจอง่ายใน $O(\ln N)$
- พอใส่ตัวใหม่เข้าไปก็ดูว่าควรจะใส่กลับไปต้นไหน แล้วขยับหรือสลับตำแหน่งตัวที่อยู่ตรงรอยต่อ (max ของต้นแรกกับ min ของต้นสอง, max ของต้นสองกับ min ของต้นสาม)
- ใช้ Policy based data structures ใน C++ ก็ได้



Route

- Summary

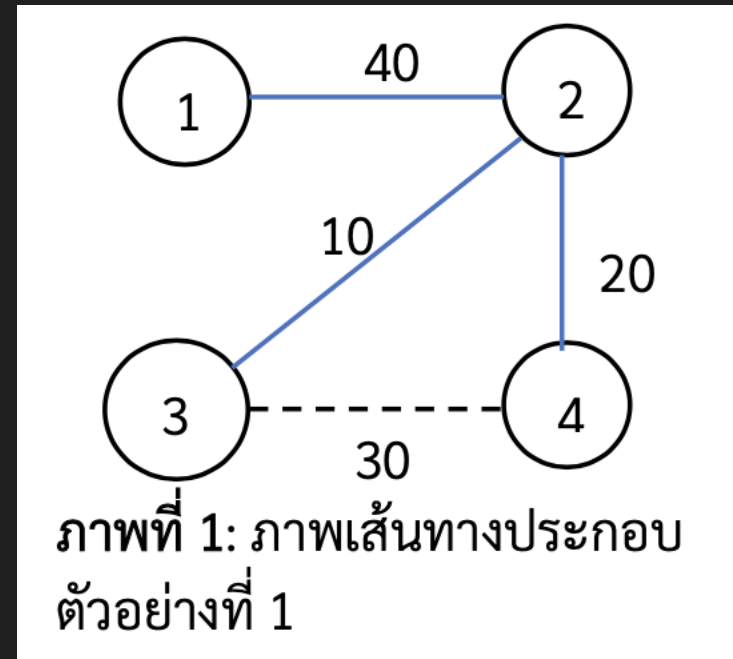
- constructive
- ส่งแบบ function
- คิดคะแนนแบบ formula
- วางเส้นเชื่อมเพื่อให้ได้ MST ที่มีค่ามากที่สุด

- Basic Solution

- ต่อยังไงก็ได้ให้ตรงเงื่อนไข (ไม่ซ้ำ และ connected จะต่อ (1,2), (1,3), (1,4)... ก็ได้
- ได้คะแนนนิดนึงแน่นอน

- Brute Force Solution $O(\text{ทุก permutation ของ edges} * \text{อะไรไม่รู้})$

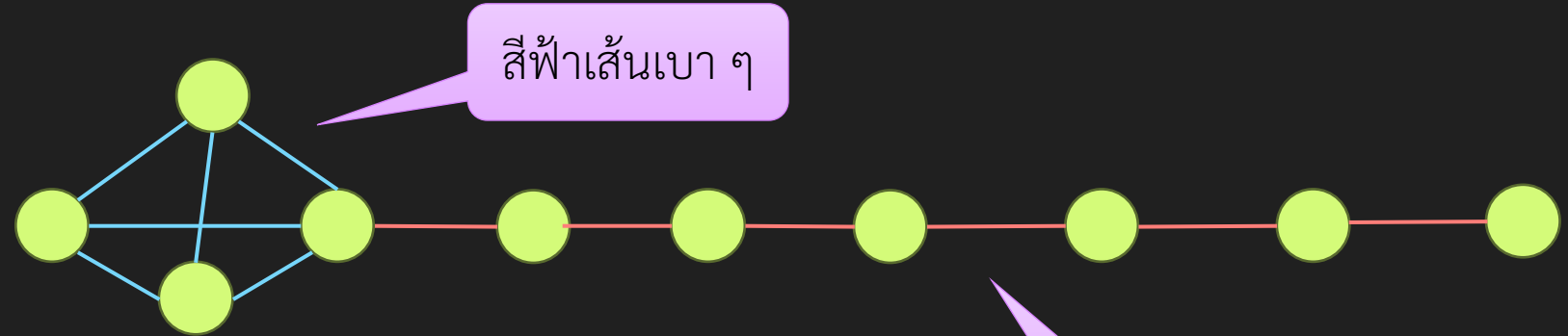
- ลองทุก permutation ของการเลือกเส้นเชื่อม



Route

- Official Solution

- ข้อสังเกต: MST พยายามเลือกเส้นหนักร้อย ๆ แต่เราอยากให้ MST มีน้ำหนักเยอะ ๆ ก็ต้องบังคับให้เลือกเส้นที่หนัก ๆ
- เป้า: อยากได้กราฟรูปแบบนี้



- เรียงเส้นเชื่อมจากน้ำหนักน้อยไปมาก (input เรียงไว้ให้แล้ว)
- ไล่เติมเส้นเชื่อมสำหรับน้ำหนักน้อยไปมากตามลำดับ complete graph
 - (2,1)
 - (3,2) (3,1)
 - (4,3) (4,2) (4,1)
 - เหลือ k เส้น กับ k ปมเมื่อไรก็ต่อเป็นเส้นตรงเพื่อให้เส้นหนัก ๆ เหล่านั้นถูกบังคับเลือกใช้

ต่อเส้นเบา ๆ เชื่อมกันเยอะ ๆ เพื่อให้เส้นที่โดน MST ตัดทิ้งคือเส้นเบา ๆ

เส้นหนัก ๆ ให้เชื่อมเป็นเส้นตรง จะได้ไม่โดนตัด

Sculpture

- **Summary**

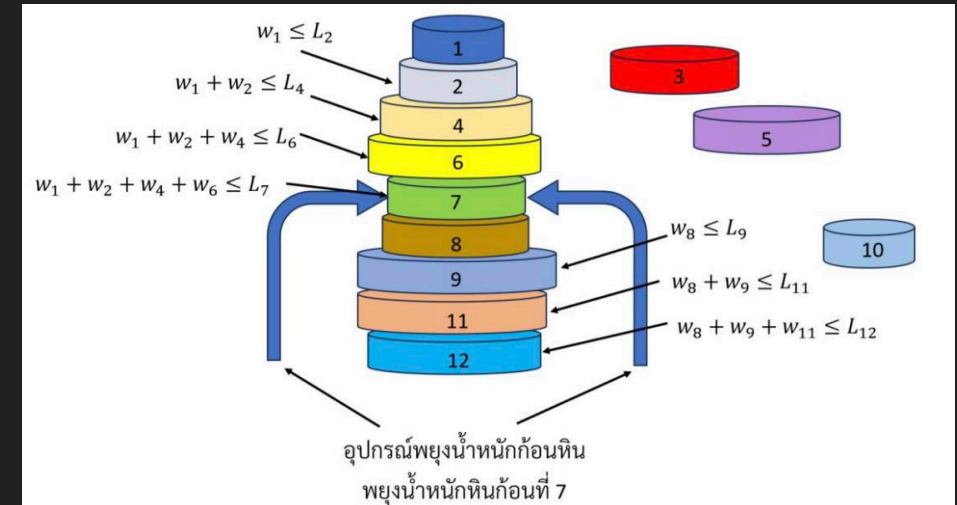
- Dynamic Programming
- Subset Sum
- Longest Increasing Subsequence variation

- **Base Solution** (subtask 1 : 13 points)

- ทดลองวางก้อนหินไปด้วย State Search ปกติ (วางจากเลขน้อยต่อไปยังเลขมาก) โดยจดจำค่าน้ำหนักรวม, จำนวนก้อนหินที่วาง แล้วมาพิจารณาว่าตัวต่อไปจะวางกลุ่มเหล่านี้ได้หรือไม่โดยเอาค่ารับน้ำหนักกีดทับของหินตัวที่จะมาต่อไปมาเปรียบเทียบกับ
- เพิ่มเงื่อนไขในการ Search โดยกำหนดว่าหินที่ใส่มาจะเอาเครื่องยกมากหรือน้อยหรือไม่เข้าไปด้วย โดยถ้ายกแล้ว ค่าน้ำหนักรวมจะกลายเป็น 0 แล้วก็เลือกวางต่อไปได้

- **Base Solution** 2 (subtask 2 : 19 points)

- จาก Sol1 ถ้าทำการบันทึกผลที่เคย Search มาไว้ก่อนแล้วค่อย lookup ก็จะเหมือนการทำ dynamic programming แบบง่าย ๆ



Sculpture

- **Basic Solution 3** (Dynamic Programming (Subsetsum approach) : 7 points)

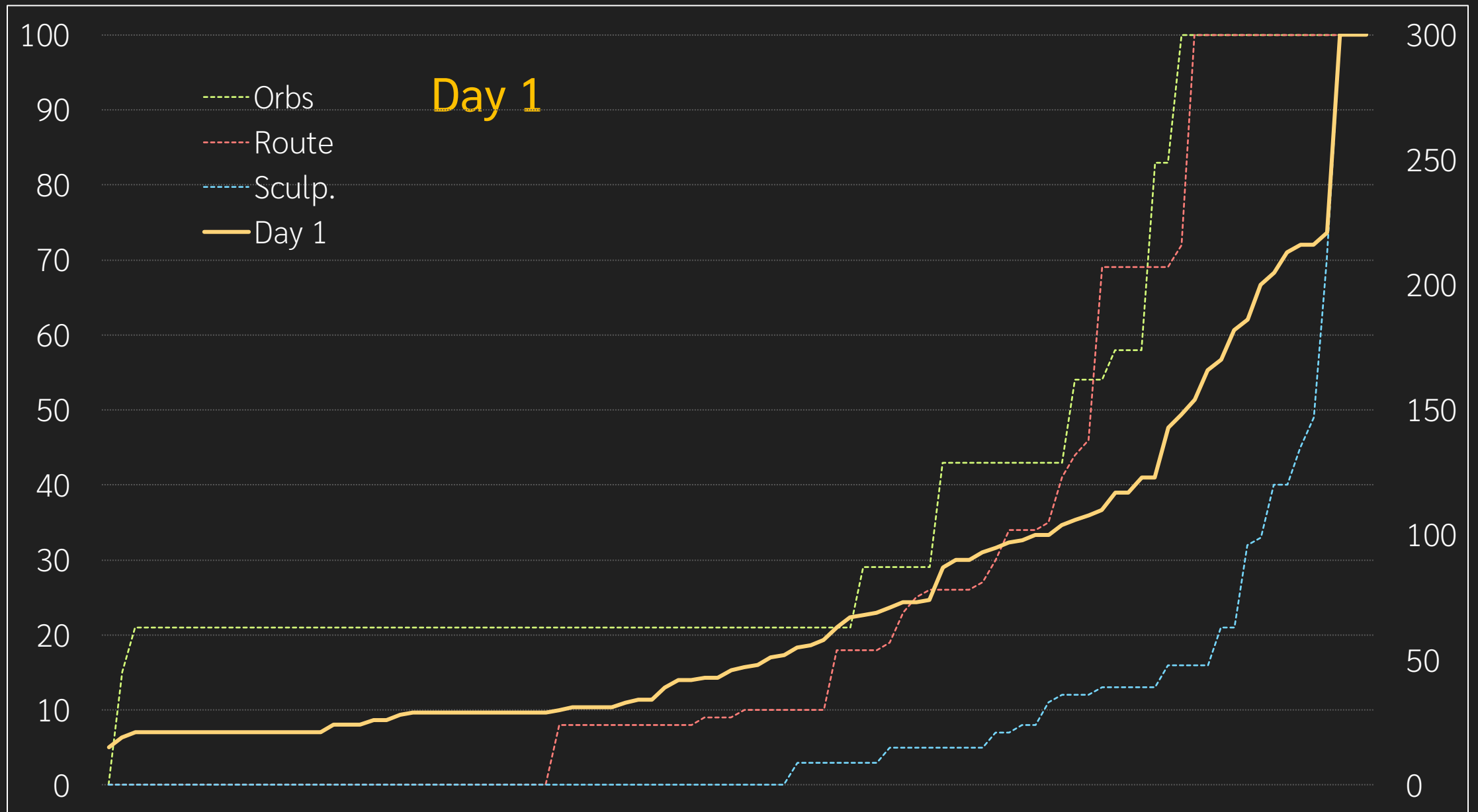
- พิจารณาคำถามลำดับ โดยดูว่าในหินก่อนหน้าที่ผ่านมา k ก้อน จะสามารถสร้างกองหินได้น้ำหนักรวมเท่าไรบ้าง และมีจำนวนหินที่รวมแล้วได้น้ำหนักดังกล่าวมากที่สุดกี่ก้อน จากนั้นพิจารณาความสามารถรับน้ำหนักหินที่มีแล้วก็ทำการสร้างกองหินใหม่ขึ้นมา
- คำตอบที่ได้หลังจากเลือกกองหินเริ่มต้นเป็นก้อนที่ i และเลือกไปถึงก้อนที่ j หมายถึงการเริ่มวางกองหินโดยใช้หิน i และพิจารณาไปจนถึง j จะได้หินมากที่สุดกี่ก้อน
- ทำการเปลี่ยนการเลือกหินเริ่มต้นไปเพื่อหาว่าถ้าจะสร้างกองหินด้านล่างจะได้เท่าไร
- $O(N^4)$ โดยประมาณ

- **Basic Solution 4** (Dynamic Programming (LIS approach) : 23 points)

- พิจารณาคำถามลำดับ โดยเปลี่ยนเป็นว่า เพื่อสร้างกองหิน K ก้อนจะใช้น้ำหนักรวมน้อยสุดเท่าไรแทน
- กรณีที่กองหิน i สามารถรับรองน้ำหนักกองหินที่มี $K-1$ ก้อนได้ จะได้ว่า
- $dp[K] = \min(dp[K-1] + W[i], dp[K])$
- คำตอบที่ได้หลังจากเลือกกองหินเริ่มต้นเป็นก้อนที่ i และเลือกไปถึงก้อนที่ j หมายถึงการเริ่มวางกองหินโดยใช้หิน i และพิจารณาไปจนถึง j จะได้หินมากที่สุดกี่ก้อน
- ทำการเปลี่ยนการเลือกหินเริ่มต้นไปเพื่อหาว่าถ้าจะสร้างกองหินด้านล่างจะได้เท่าไร
- $O(N^3)$

Sculpture Official Solution 4 (Dynamic Programming (LIS approach))

- ทำเหมือน Solution 3 แต่ทำแค่ 1 ถึง N ครั้งเดียว และบันทึกไว้ว่า ถ้าสร้างถึงก้อนหินใด ๆ จะได้ความสูงเท่าไร $O(N^2)$
- ทำการเลือกก้อนหินจากก้อนที่ N ย้อนกลับไปถึง 1 โดยหาว่าถ้าสร้างก้อนหิน K ก้อนแล้วจะมีความสามารถรับน้ำหนักเหลือให้หินก้อนอื่นวางต่อได้มากเท่าไร แทนและบันทึกไว้ว่าถ้าเลือกถึงหินก้อนใด จะสามารถสร้างหินได้สูงสุดเท่าไร $O(N^2)$
 - สมการหน้าถัดไป
- เอาผลลัพธ์จากทั้งสองตัวมาหาคำตอบสุดท้าย $O(N)$
- กรณีที่กองก้อนหินจาก N ถึง $i-1$ สามารถรับรอน้ำหนักได้ C_{i-1}
- หินก้อน i มีน้ำหนัก และความสามารถรับรอน้ำหนักได้เป็น W_i, L_i
- จะสามารถวางหินบนกองนี้ได้เมื่อ $W_i \leq C_{i-1}$
- หากกำหนด ให้ $C[K]$ คือค่ารองรับน้ำหนักสูงสุดเมื่อมีหินกองอยู่ K ก้อน จะได้ว่า
- $C[K] = \max(\min([C[K-1] - W_i, L_i], C[K])$



Score Summary – Day 1

โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Orbs	15	95	3,776	817
Route	14	62	2,639	594
Sculpture	3	44	918	609

ไม่มีคนไม่ได้คะแนน

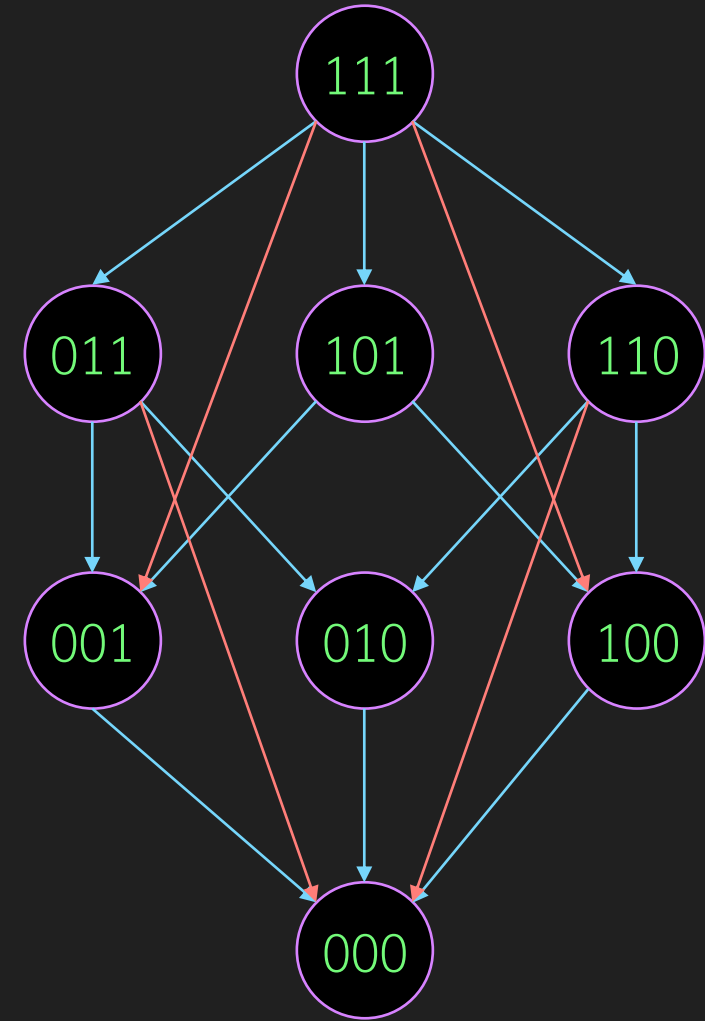
Bit string

- Summary

- ทดสอบพื้นฐาน
- Implicit Graph
- Classic DP (หา longest path ใน directed acyclic graph)

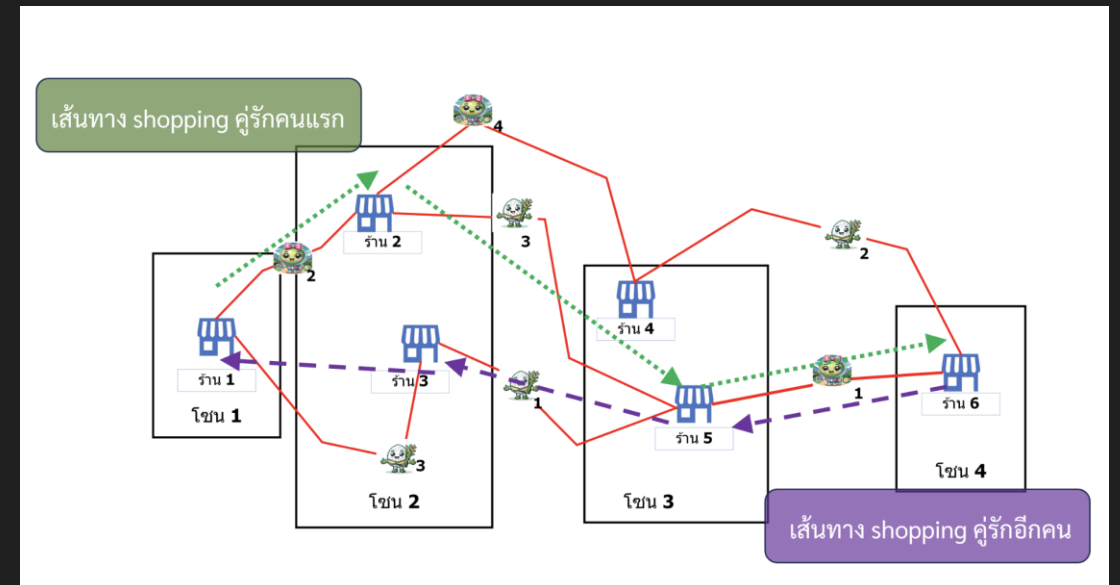
- Official Solution $O(kN^3)$

- ให้สังเกตว่าเส้นเชื่อมจะเชื่อมจากปมที่มีจำนวน 1 มากกว่าไปยังปมที่มีจำนวน 1 น้อยกว่า
 - ดังนั้นเป็น Directed Acyclic Graph ใช้ Dynamic Programming ได้
- Partial Solution ให้ใช้ BFS หรือ Shortest Path
- หรือจะ Brute Force ก็พอได้บ้าง



Tour Base Solution $O(N^2)$

- คำนวณผลรวมของ “ส้มโอหวาน” และ “ข้าวสารขาว” ของทุก ๆ Path จาก โซน 1 ไป L (ซึ่งก็รวม path จาก Zone L กลับมา 1 ด้วย เพราะ Path ไม่มีทิศทาง)
- แต่ละ path จะมีค่าผลรวม F, G กำกับอยู่ โจทย์คือหาคู่ (F,G) และ (F',G') ที่ “ใกล้เคียงที่สุด”
 - สูตรค่าความต่างใจคือ “ระยะห่างกำลังสอง”
- แปลง Path ทั้งหมดเป็น “พิกัดสองมิติ” ด้วยพิกัด (F,G)
 - ต้อง Recursive สร้าง Path ทั้งหมด
 - วัฏวิธี implementation ด้วย
- Official Solution $O(N \lg N)$
 - เขียน Closest Pair ด้วย Divide & Conquer
 - Textbook D&C แต่เขียนยากหน่อย

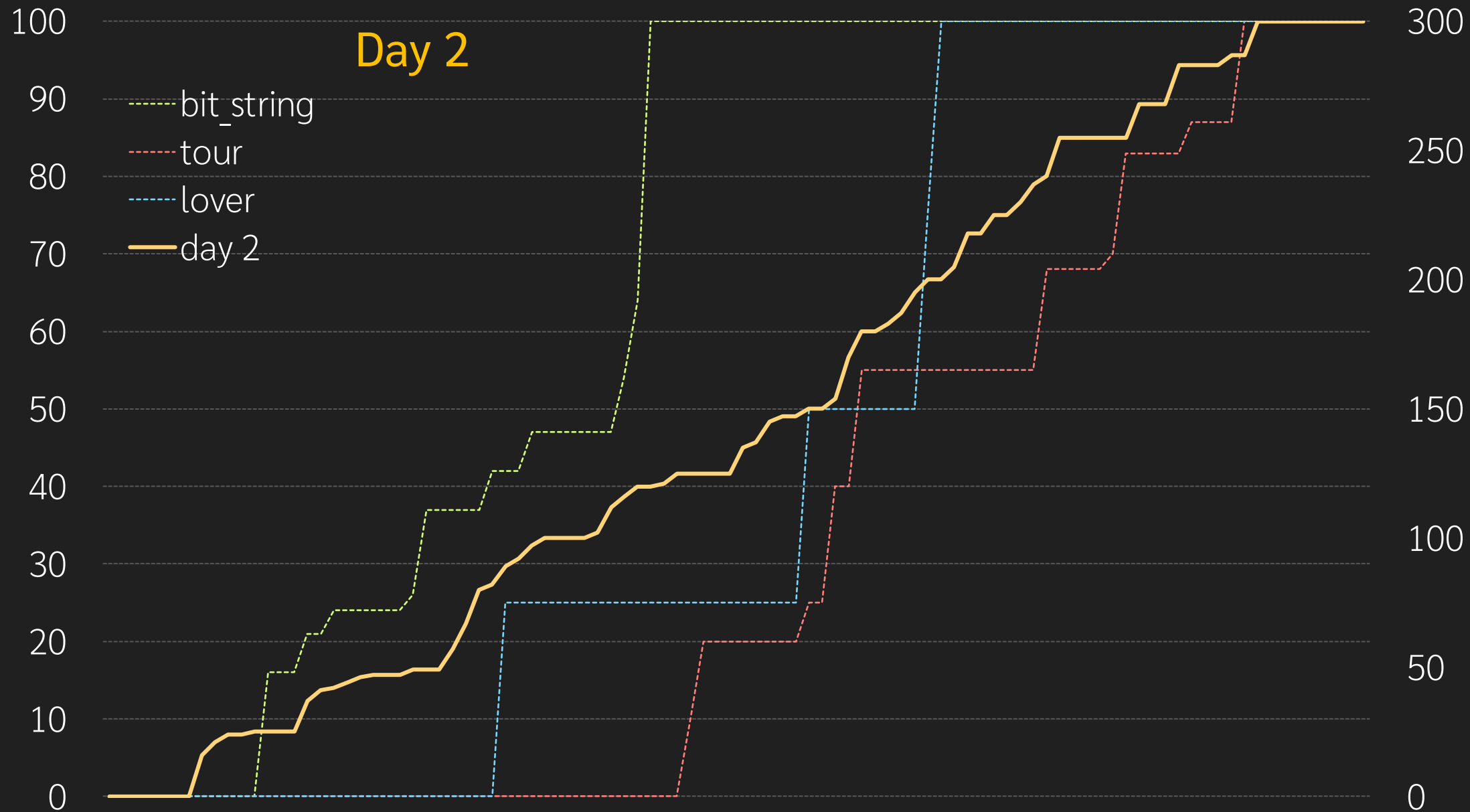


Lover

- Summary
 - Dynamic Programming
- ให้อาเรย์มา อยากรู้ว่าแบ่งเป็น M ส่วนที่ตรงตามเงื่อนไขได้หรือไม่
 - อยากรู้ว่า M น้อยสุดที่ทำได้คืออะไร
- Base Solution $O(N^3)$
 - ให้ $OK[a][i]$ (เป็น true/false) คือบอกว่า เราสามารถแบ่งช่องที่ 1 ถึง i เป็น a ช่วงพักได้
 - คำตอบคือหา x น้อยสุดที่ $OK[x][N]$ เป็น true
 - $OK[a][b] = OK[a-1][x] \ \&\& \ Range[x][b]$ สำหรับ x ตั้งแต่ a ถึง b
 - ให้ $Range[L][R]$ แปลว่า “ช่วง L ถึง R ” มีค่าใช้จ่ายรวม (ของทั้งคู่รัก) ไม่เกิน V, W
 - $Range[L][R]$ คำนวณเอาตรง ๆ เลย

Lover Official Solution $O(N^2)$

- ลดรูปจาก DP เดิม (คือ $OK[a][b]$)
- เพราะ $OK[a][b]$ ใช้ $O(N)$ แล้วต้องคำนวณ $N * N$ ช่องเลยเป็น $O(N^3)$
- ให้ $DP[i] = \min \{ x \text{ ที่ทำให้ } OK[x][i] \text{ เป็นจริง} \}$ หรือก็คือ $DP[i]$ คือค่า M น้อยสุดที่ทำได้เมื่อคิดเฉพาะช่อง 1 ถึง i
- สังเกตว่า $DP[j] = \min(DP[i] + 1 \text{ หรือ } Range[i+1][j])$ ก็เลยหา $DP[j]$ ได้ใน $O(N)$
- คำตอบคือ $DP[N]$



Score Summary – Day 2

โจทย์	คนเต็ม	คนได้คะแนน	คะแนนรวม	จำนวน sub
Bit String	55	84	6,518	769
Tour	10	52	3,243	1,307
Lover	33	66	4,400	749

มีคนไม่ได้คะแนน 7 คน

TOI 20 Summary

